

SkyvoltRobot MVP

Phase 3 보고서 — 매니퓰레이터 + 플러그인

작성일: 2026 - 06 - 15

1. Phase 3 개요

Phase 3의 목표는 충전 플러그를 실제로 삽입하는 매니퓰레이터를 갖추는 것이다. 로드맵의 4개 작업 중 두 개의 핵심 IP — 텔레스코픽 리드스크류 기구학과 RGB - D 충전구 자세 추정 — 을 순수 파이썬으로 구현했고, 둘은 곧바로 연결된다(자세 → 삽입 계획). 나머지(비주얼 서보잉·힘피드백)는 제어 루프가 필요해 후속으로 진행한다.

작업	성격	상태
텔레스코픽 리드스크류 기구학	순수 SW	기구학 완료
비전 — RGB - D 충전구 자세 검출	순수 SW	자세추정 완료
비주얼 서보잉 (플러그 삽입)	순수 SW	페루프 완료
힘/토크 정합 피드백	순수 SW	감지 완료

4개 작업의 핵심 IP를 모두 구현. skyvolt_manipulator + skyvolt_vision, 테스트 46개, 저장소 전체 149개 통과. 검출 → 자세추정 → 정렬·삽입·도킹 → 힘 기반 안전감시가 연결된다. 남은 것은 Gazebo 센서 연동.

2. 텔레스코픽 리드스크류 기구학

SkyvoltRobot 충전 플러그는 두 리드스크류 액추에이터로 구동된다(논문 4.2.3.5절): Y축 텔레스코프가 플러그를 차량 충전구로 뺀고(스트로크 320 mm), X축 래치가 커넥터를 체결한다(스트로크 40 mm). 리드스크류는 모터 회전을 직선 운동으로 바꾼다 — 1회전이 나사 리드만큼 전진하므로 모터각↔스트로크가 선형으로 매핑되고, 리드로 정해지는 토크→추력 기계적 이득과 스트로크 한계를 갖는다.

LeadScrew — 단축 액추에이터

- 회전수 ↔ 변위 선형 매핑, 모터각, 스트로크 한계 클램핑
- 선속도 ↔ rpm, 두 변위 간 이동 시간
- 토크 → 추력: $F = 2\pi \cdot \eta \cdot T / \text{lead}$ (리드가 작을수록 추력↑, 속도↓)

TelescopicManipulator — 2축 삽입 계획

- 텔레스코프(reach) → 래치(engage) 순차 삽입 계획: 축별 회전수·시간·총 시간 산출
- 각 축 스트로크 초과 목표는 사유와 함께 거부(feasible=False)
- 기본값이 URDF 조인트 속도 한계(텔레스코프 0.1 m/s, 래치 0.05 m/s)와 일치

3. 리드스크류 기구학 검증 (테스트 14개)

핵심 기구학 관계를 모두 테스트로 고정했다 — 변위 선형성·역함수, 스트로크 클램핑, 선속도·이동시간, 토크→추력(리드 작을수록 추력 큼)·효율, 삽입 계획의 타이밍과 스트로크 초과 거부.

```
>>> m = TelescopicManipulator()
>>> p = m.plan_insertion(reach_m=0.30, engage_m=0.03)
>>> p.feasible, round(p.reach_time_s,1), round(p.total_time_s,1)
(True, 3.0, 3.6)          # 0.30 m @ 0.1 m/s + 0.03 m @ 0.05 m/s
>>> m.plan_insertion(reach_m=0.50, engage_m=0.03).feasible
```

4. 비전 — RGB-D 충전구 6-DoF 자세 추정

플러그를 삽입하려면 차량 충전구의 위치를 알아야 한다. RGB-D 센서가 주는 충전구 특징점(소켓 fiducial)의 3D 위치를 충전구 자체 좌표의 모델 점과 정합해 6-DoF 자세를 구한다. Kabsch/Umeyama 강제 최소제곱을 순수 파이썬+numpy로 구현했다 — 반사 방지로 항상 올바른 회전($\det = +1$)을 보장하고, 적합 잔차(RMSE)로 검출 신뢰도를 함께 낸다.

- estimate_pose / ChargingPort.detect — 특징점 대응에서 회전+병진을 복원, 신뢰도(RMSE) 반환.
- approach_distance / lateral_offset — 추정 자세를 매니퓰레이터 입력으로 변환: 접근 거리(텔레스코프 reach)와 정렬 오차(서보가 0으로 만들 목표).
- detector.py (sim 연동) — Gazebo RGB-D 영상에서 어두운 소켓 4코너를 검출해 깊이·내부파라미터로 3D 역투영. 영상→자세 연결고리. 합성 RGB-D로 영상→검출→자세→서보→도킹 검증.

5. 통합 — 비전 → 매니퓰레이터

추정한 충전구 자세가 곧바로 삽입 계획으로 흘러간다(두 IP 모듈의 결합).

```
charging-port detect RMSE : 0.38 mm (high confidence)
approach distance (reach) : 280 mm
lateral offset           : 19.9 mm (servo target -> 0)
plan_insertion           : feasible=True, total 3.4 s
```

6. 비주얼 서보잉 — 정렬 → 삽입 → 도킹 페루프

추정된 충전구 자세를 입력으로 매니퓰레이터를 페루프 제어한다. 2단계 제어기 — ALIGN(비례 횡방향 정렬로 정렬오차를 허용치까지 축소) → INSERT(텔레스코프 전진으로 삽입) → DOCKED, 정해진 틱 안에 수렴 못 하면 FAULT. 순수 파이썬 제어를 운동학 플랜트 대상 페루프로 검증했다.

```
initial : lateral 50.0 mm, distance 300 mm
result  : DOCKED in 6.8 s, final lateral 1.96 mm (< 2 mm tol)
```

단위 테스트 9개: 정렬 수렴·단조감소, 위상 전진성, 명령 한계, latch 1회, 미수렴 FAULT, 터미널 고착, 비전→서보 end-to-end 도킹.

7. 힘/토크 정합 피드백

삽입 중 손목 F/T 센서의 접촉 렌치로 오정렬을 감지한다. 정렬된 삽입은 거의 순수 축방향 힘이지만, 어긋난 플러그는 소켓 림에 눌러 횡력·코킹 모멘트를 낸다. InsertionForceGuard가 렌치를 FREE/SEATING/MISALIGNED/OVERLOAD로 분류해 서보 루프에 CONTINUE/CORRECT(후퇴·재정렬)/ABORT(중단)를 권고한다. 판정은 횡력/축력 비, 코킹 모멘트, 안전 한계로 한다. 단위 테스트 8개.

8. 남은 작업 (계획)

- Gazebo 연동 (잔여) — 접촉 물리 sim(F/T 실데이터), 실제 카메라 영상 프레임·검출 튜닝, 매니퓰레이터 조인트 구동. (핵심 IP·RGB-D 센서·특징 검출 프론트엔드는 완료.)

9. Phase 3 추가 파일

파일	구분	내용
skyvolt_manipulator/ (패키지)	신규	리드스크류 기구학 + 힘/토크 가드 + 22 테스트

skyvolt_vision/ (패키지)	신규	자세 추정 + 서보잉 + RGB-D 특징 검출 + 24 테스트
pytest.ini / run_tests.sh	수정	매니플레이터·비전 테스트 경로 추가

10. 실행 방법

```
cd skyvolt_mvp
./scripts/run_tests.sh    # all tests (149, incl. 22 manip + 24 vision)
python3 -c "from skyvolt_vision import ChargingPort, VisualServo, \
simulate_insertion as sim; ..." # detect -> servo -> dock
```

11. 결론

- Phase 3의 네 핵심 IP를 순수 파이썬으로 구현·검증했다 — 리드스크류 기구학, 충전구 6-DoF 자세 추정, 비주얼 서보잉, 힘/토크 정합 피드백.
- 검출 → 자세추정 → 정렬·삽입·도킹 → 힘 기반 안전감시가 하나의 파이프라인으로 연결된다. 남은 것은 Gazebo 센서 연동이다.